ETH Zürich

Informatik I
Übungsstunde 04

Johannes Odenthal

ETZ J 91
cs.jodenthal.com

Feedback · Exercise 2

Vorher

```
int a;
int b;
std::cin >> a;
std::cin >> b;
int c;
c = a * b;
```

Lesbarer

```
int width, height;
std::cin >> width >> height;
int area = width * height;
```

Aufgabenstellung genau lesen; Variablen nach ihrer Bedeutung benennen.

Feedback · Exercise 3

`a` und `b` sind vom Typ `bool`.

Ausführlich

```
b == true

b == false

(a && !b) || (!a && b)
```

Direkt

```
b

!b

a != b
```

XOR: Genau einer der beiden Werte ist `true`.

Sauber formatieren und Vorlagenreste entfernen.

Hausaufgabenfragen

Fragen zu Exercise 2 oder 3?

Loops

Loop Correctness

`n` ist eine ganze Zahl vom Typ `int`.

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Für welche Werte von `n` unterscheiden sich die Ausgaben?

Was passiert bei n = 0?

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Was passiert bei n = 0?

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Eingabe	for	while	do–while
`n = 3`	1, 2, 3	1, 2, 3	1, 2, 3
`n = 0`	keine Ausgabe	keine Ausgabe	1

Was passiert bei n = INT_MAX?

`INT_MAX` ist der grösste darstellbare Wert von `int`.

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Was passiert bei n = INT_MAX?

`INT_MAX` ist der grösste darstellbare Wert von `int`.

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Integer-Überlauf Nach der letzten Ausgabe erhöht `++i` nochmals.

Was passiert bei n = INT_MAX?

`INT_MAX` ist der grösste darstellbare Wert von `int`.

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Integer-Überlauf Nach der letzten Ausgabe erhöht `++i` nochmals.

Stoppt korrekt `++i` erreicht `INT_MAX`; danach ist `i < n` falsch.

Was passiert bei n = INT_MAX?

`INT_MAX` ist der grösste darstellbare Wert von `int`.

for

```
for (int i = 1;
     i <= n; ++i) {
  std::cout << i
            << "\n";
}
```

Integer-Überlauf Nach der letzten Ausgabe erhöht `++i` nochmals.

while

```
int i = 0;
while (i < n) {
  std::cout << ++i
            << "\n";
}
```

Stoppt korrekt `++i` erreicht `INT_MAX`; danach ist `i < n` falsch.

do–while

```
int i = 1;
do {
  std::cout << i++
            << "\n";
} while (i <= n);
```

Integer-Überlauf `i++` erhöht bereits bei der letzten Ausgabe.

Vorzeichenbehafteter Integer-Überlauf: undefined behavior.

for ⟶ while

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
for (int i = 0; i < n; ++i) {
  std::cout << i << " ";
}
```

Lösung

`n = 3`: Ausgabe `0 1 2`

for ⟶ while

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
for (int i = 0; i < n; ++i) {
  std::cout << i << " ";
}
```

Lösung

```
{
  int i = 0;
  while (i < n) {
    std::cout << i << " ";
    ++i;
  }
}
```

Initialisierung davor · Bedingung im Kopf · Update am Rumpfende

`n = 3`: Ausgabe `0 1 2`

while ⟶ for

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
int i = n;
while (i > 0) {
  std::cout << i << " ";
  --i;
}
```

Lösung

`n = 3`: Ausgabe `3 2 1`

while → for

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
int i = n;
while (i > 0) {
  std::cout << i << " ";
  --i;
}
```

Lösung

```
int i = n;
for (; i > 0; --i) {
  std::cout << i << " ";
}
```

`i` bleibt ausserhalb deklariert; Bedingung und Update stehen im Kopf.

`n = 3`: Ausgabe `3 2 1`

do–while ⟶ for

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
int i = 0;
do {
  std::cout << i << " ";
  ++i;
} while (i < n);
```

Lösung

`n = 0`: Ausgabe `0` · `n = 3`: Ausgabe `0 1 2`

do–while ⟶ for

Die Ausgabe soll für dasselbe `n` gleich bleiben.

Ausgangscode

```
int i = 0;
do {
  std::cout << i << " ";
  ++i;
} while (i < n);
```

Lösung

```
int i = 0;
std::cout << i << " ";
++i;
for (; i < n;) {
  std::cout << i << " ";
  ++i;
}
```

Erster Durchlauf vor dem Test; die `for`-Schleife übernimmt den Rest.

`n = 0`: Ausgabe `0` · `n = 3`: Ausgabe `0 1 2`

Fragen?

Computing Taylor Series

Sinus als unendliche Summe

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Was ändert sich von Summand zu Summand?

Sinus als unendliche Summe

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Was ändert sich von Summand zu Summand?

Vorzeichen: $+, -, +, -, \dots$

Sinus als unendliche Summe

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Was ändert sich von Summand zu Summand?

Vorzeichen: $+, -, +, -, \dots$

Exponent: $1, 3, 5, 7, \dots$ — jeweils $+2$

Sinus als unendliche Summe

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Was ändert sich von Summand zu Summand?

Vorzeichen: $+, -, +, -, \dots$

Exponent: $1, 3, 5, 7, \dots$ — jeweils $+2$

Nenner: $1!, 3!, 5!, 7!, \dots$ — Fakultät des Exponenten

Warmup: Fakultät

$$n! = \prod_{k=1}^{n} k$$

Berechne die Fakultät einer nichtnegativen ganzen Zahl `n`.

Beispiel: `n = 5` ⟶ `120`

Welche Variablen und Startwerte brauchen wir?

Warmup: Fakultät

$$n! = \prod_{k=1}^{n} k$$

Berechne die Fakultät einer nichtnegativen ganzen Zahl `n`.

Beispiel: `n = 5` ⟶ `120`

Welche Variablen und Startwerte brauchen wir?

Produkt `result = 1.0` · Zähler `k = 1`

Bei `n = 0`: kein Durchlauf, Ergebnis `1`.

Fakultät mit einer Schleife

```
double result = 1.0;
```

Start: Produkt ist `1`, nicht `0`.

Fakultät mit einer Schleife

```
double result = 1.0;
for (int k = 1; k <= n; ++k) {

}
```

Start: Produkt ist `1`, nicht `0`.

Zähler: von `1` bis einschliesslich `n`.

Fakultät mit einer Schleife

```
double result = 1.0;
for (int k = 1; k <= n; ++k) {
  result *= k;
}
```

Start: Produkt ist `1`, nicht `0`.

Zähler: von `1` bis einschliesslich `n`.

Update: das bisherige Produkt mit `k` multiplizieren.

Fakultät mit einer Schleife

```
double result = 1.0;
for (int k = 1; k <= n; ++k) {
  result *= k;
}
```

Start: Produkt ist `1`, nicht `0`.

Zähler: von `1` bis einschliesslich `n`.

Update: das bisherige Produkt mit `k` multiplizieren.

$$5!: \quad 1 \to 1 \to 2 \to 6 \to 24 \to 120$$

Die Fakultät als Funktion

```
double factorial(int n) {
  double result = 1.0;
  for (int k = 1; k <= n; ++k) {
    result *= k;
  }
  return result;
}
```

```
double value = factorial(5);  // value: 120
```

Taylor Series · CodeExpert

Berechne `sin(x)` mit einer Genauigkeit von `0.000001`.

$$\sin x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

`x` wird im Bogenmass eingegeben.

Eingabe	Ausgabe
`1`	`0.841471`

09:00–09:15

Pause

Taylor Series · CodeExpert

Berechne `sin(x)` mit einer Genauigkeit von `0.000001`.

$$\sin x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

`x` wird im Bogenmass eingegeben.

Eingabe	Ausgabe
`1`	`0.841471`

Taylor Series · Mit Papier überlegen

Für `x = 1`: Berechne die ersten drei Summanden und ihre Summe.

- Welche Grössen bleiben konstant, welche ändern sich?
- Mit welchen Werten starten Summe und Zähler?
- Woran erkennst du, dass ein Summand klein genug ist?

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Taylor Series · Mit Papier überlegen

Für `x = 1`: Berechne die ersten drei Summanden und ihre Summe.

- Welche Grössen bleiben konstant, welche ändern sich?
- Mit welchen Werten starten Summe und Zähler?
- Woran erkennst du, dass ein Summand klein genug ist?

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

$$1 - \frac{1}{6} + \frac{1}{120} = \frac{101}{120} \approx 0.841667$$

Taylor Series · Mit Papier überlegen

Für `x = 1`: Berechne die ersten drei Summanden und ihre Summe.

- Welche Grössen bleiben konstant, welche ändern sich?
- Mit welchen Werten starten Summe und Zähler?
- Woran erkennst du, dass ein Summand klein genug ist?

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

$$1 - \frac{1}{6} + \frac{1}{120} = \frac{101}{120} \approx 0.841667$$

`x` bleibt konstant; Index, Summand und Summe ändern sich.

Start: Summe `x`, nächster Index `n = 1` · Stopp: $|\text{term}| \leq 10^{-6}$

Taylor Series · Partnerarbeit

Implementiert eure Lösung in CodeExpert.

- Vergleicht zuerst eure Überlegungen auf Papier.
- Verwendet die Fakultätsfunktion und `std::pow`.
- Addiert Summanden, bis ihr Betrag höchstens `0.000001` ist.
- Testet `x = 0`, `x = 1` und `x = −1`.

Kontrolle: `sin(1)` ≈ `0.841471` · `sin(−1)` ≈ `−0.841471`

Fragen?

Scopes

Fehlersuche: Scopes

Dieser Code enthält einen Fehler.

```
int x = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Warum kompiliert dieser Code nicht?

Fehlersuche: Scopes

Dieser Code enthält einen Fehler.

```
int x = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Warum kompiliert dieser Code nicht?

`a` ist nach dem Ende des `if`-Blocks nicht sichtbar.

Fehlersuche: Scopes

Dieser Code enthält einen Fehler.

```
int x = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Warum kompiliert dieser Code nicht?

`a` ist nach dem Ende des `if`-Blocks nicht sichtbar.

Die letzte Ausgabe in den Block verschieben — oder `a` ausserhalb deklarieren.

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Schritt	Auswertung	a_1	a_2	Ausg.

a_1: aussen · a_2: innen

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	

a_1: aussen · a_2: innen

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	

a_1: aussen · a_2: innen

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	
Test	2 < 7 ⟶ true	2	—	

a_1: aussen · a_2: innen

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	
Test	2 < 7 ⟶ true	2	—	
Init	inneres a = 8	2	8	

a_1: aussen · a_2: innen

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

a_1: aussen · a_2: innen

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	
Test	2 < 7 ⟶ true	2	—	
Init	inneres a = 8	2	8	
Ausgabe	inneres a lesen	2	8	8

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

a_1: aussen · a_2: innen

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	
Test	2 < 7 ⟶ true	2	—	
Init	inneres a = 8	2	8	
Ausgabe	inneres a lesen	2	8	8
Blockende	inneres a endet	2	—	8

Shadowing: Welche Ausgabe entsteht?

```
int x = 2;
int a = 2;
if (x < 7) {
  int a = 8;
  std::cout << a;
}
std::cout << a;
```

a_1: aussen · a_2: innen

Schritt	Auswertung	a_1	a_2	Ausg.
Init	x = 2	—	—	
Init	äusseres a = 2	2	—	
Test	2 < 7 ⟶ true	2	—	
Init	inneres a = 8	2	8	
Ausgabe	inneres a lesen	2	8	8
Blockende	inneres a endet	2	—	8
Ausgabe	äusseres a lesen	2	—	82

Scopes in einer Schleife

```
int sum = 0;
for (int i = 0; i < 5; ++i) {
  int a;
  std::cin >> a;
  sum += a;
}
```

Wo sind die Namen sichtbar?

Scopes in einer Schleife

```cpp
int sum = 0;
for (int i = 0; i < 5; ++i) {
  int a;
  std::cin >> a;
  sum += a;
}
```

Wo sind die Namen sichtbar?

sum · auch nach der Schleife

Scopes in einer Schleife

```
int sum = 0;
for (int i = 0; i < 5; ++i) {
  int a;
  std::cin >> a;
  sum += a;
}
```

Wo sind die Namen sichtbar?

sum · auch nach der Schleife

i · Kopf und Rumpf der Schleife

Scopes in einer Schleife

```
int sum = 0;
for (int i = 0; i < 5; ++i) {
  int a;
  std::cin >> a;
  sum += a;
}
```

Wo sind die Namen sichtbar?

sum · auch nach der Schleife

i · Kopf und Rumpf der Schleife

a · ab Deklaration im Rumpf

Bei jedem Durchlauf entsteht ein neues `a`.

Fragen?

Nachschlagen

Rückgabewert und Ausgabe

Eine Zahl zurückgeben

```
double area(double w,
            double h) {
  return w * h;
}
```

```
double result = area(3.0, 4.0);
```

Eine Zahl ausgeben

```
void print_value(double value) {
  std::cout << value << "\n";
}
```

```
print_value(12.0);
```

break und continue

break

```
for (int i = 1; i <= 5; ++i) {
  if (i == 3) {
    break;
  }
  std::cout << i;
}
```

continue

```
for (int i = 1; i <= 5; ++i) {
  if (i == 3) {
    continue;
  }
  std::cout << i;
}
```

Welche Ausgabe entsteht jeweils?

break und continue

break

```
for (int i = 1; i <= 5; ++i) {
  if (i == 3) {
    break;
  }
  std::cout << i;
}
```

continue

```
for (int i = 1; i <= 5; ++i) {
  if (i == 3) {
    continue;
  }
  std::cout << i;
}
```

Welche Ausgabe entsteht jeweils?

break: `12` · **continue:** `1245`

switch

```
switch (door) {
  case 1:
  case 3:
    std::cout << "Wrong choice\n";
    break;
  case 2:
    std::cout << "You won\n";
    break;
  default:
    std::cout << "Unknown door\n";
}
```

Was wird bei `door = 2` ausgegeben?

switch

```
switch (door) {
  case 1:
  case 3:
    std::cout << "Wrong choice\n";
    break;
  case 2:
    std::cout << "You won\n";
    break;
  default:
    std::cout << "Unknown door\n";
}
```

Was wird bei `door = 2` ausgegeben?

Ausgabe: `You won`